# Cuda C++ RGB to grayscale conversion

> cuda

**reddy_v** Nov 26 '22

I trying to convert bgr images to grayscale & I want to round of the final result.

Here is the kernel that I am using,

```
__global__ void get_grayscale_images(int img_
                                     unsigned
                                     float *
{
  int col = blockIdx.x * blockDim.x + threadI
  int row = blockIdx.y * blockDim.y + threadI

  if(row >= img_height || col >= img_width) r

  unsigned char b = d_input_image[((row * img
  unsigned char g = d_input_image[((row * img
  unsigned char r = d_input_image[((row * img

  d_grayscale_image[row * img_width + col] =
}
}
```

For a particular pixel which has bgr values of [168, 160, 172], ideally it should have a grayscale pixel value of 165, however when I use the above kernel it is giving me 164. Is there something wrong in the kernel like datatype conversions, because the value of (float(172) * 0.299f + float(160) * 0.587f + float(168) * 0.114f) is equal to 164.956 and when we do round it should be equal to 165.

I have validated the pixel value of the input images, it is perfectly fine. And also when I hard code the values it is also working fine

**striker159** Nov 26 '22

On my calculator, `172 * 0.299 + 160 * 0.587 + 168 * 0.114 = 164.5`, which is computed in the kernel as `164.499985`. 164 should be the correct rounded result.

**njuffa** Top Contributor Nov 26 '22

> reddy_v:
>
> because the value of (float(172) * 0.299f + float(160) * 0.587f + float(168) * 0.114f) is equal to 164.956

How did you compute that? I get 164.5. For half-way cases `round` rounds away from zero, so the result is 165.

Now, when I do this on the GPU using code compiled with the CUDA 11.8 toolchain, I get 164.499985, which rounds to 164. Note that the intermediate result differs by 1 ulp from 164.5.

As opposed to math, floating-point arithmetic is not associative, so the order of operations can have an influence on the result. By default, the CUDA compiler turns on FMA contraction (same as the user specifying `-fmad=true`). In this case, it turns the computation into:

```
    gray = __fmaf_rn (float(b), 0.114f,
                      __fmaf_rn (float(r), 0.29
                                 __fmul_rn (flo
```

In general, FMA contraction is good for performance (in the above, we need only three floating-point operations instead of five), and *on average* it is also good for accuracy, as it reduces the overall rounding error and provides some protection against subtractive cancellation. But it can also cause results to differ from a desired reference result, as is the case here.

Instead of turning off FMA contraction altogether (`-fmad=false`), CUDA programmers can suppress it locally by the use of intrinsics, which the compiler will no include in FMA merging. So in this case we could code:

```
    gray = (__fmul_rn (float(r), 0.299f) +
            __fmul_rn (float(g), 0.587f) +
            __fmul_rn (float(b), 0.114f));
```

I also get the desired result for the particular case from the original post when I code this FMA-based version, but of course it could result in tiny differences for other input combinations:

```
    gray = __fmaf_rn (float(r), 0.299f,
                      __fmaf_rn (float(g), 0.58
                                 __fmul_rn (flo
```

In general, it is not a good idea to rely on bit-identical processing in floating-point computation, but I understand it may sometimes be necessary to match a "golden" reference. Not sure whether that is the case here: Does anything bad happen if the grayscale value comes out as 164 instead of 165? A human is unlikely to notice.

I am appending my little test program for reference.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

__global__ void kernel (unsigned char r, unsign
{
    float gray = (float(r) * 0.299f + float(g)
    printf ("gray=%15.8e\n", gray);
    gray = __fmaf_rn (float(b), 0.114f,
                      __fmaf_rn (float(r), 0.29
                                 __fmul_rn (flo
    printf ("gray=%15.8e\n", gray);
    gray = (__fmul_rn (float(r), 0.299f) +
            __fmul_rn (float(g), 0.587f) +
            __fmul_rn (float(b), 0.114f));
    printf ("gray=%15.8e\n", gray);
    gray = __fmaf_rn (float(r), 0.299f,
                      __fmaf_rn (float(g), 0.58
                                 __fmul_rn (flo
    printf ("gray=%15.8e\n", gray);
}

int main (void)
{
    kernel<<<1,1>>>(172, 160, 168);
    cudaDeviceSynchronize();
    return EXIT_SUCCESS;
}
```

**reddy_v** Nov 27 '22

@njuffa @striker159 , my bad its 164.5, so basically what I am trying to do is to convert the color images to grayscale images the same way opencv does, and i want these images very accurate since i have do some more image processing & convolutions stuff after this grayscale conversion.
Here are the things that I tried out,
**Step1**:

```
    unsigned char b = 168;
    unsigned char g = 160;
    unsigned char r = 172;
    d_grayscale_image[row * img_width + col] =
```

When I hard code these values in the kernel, output has a value of 165. now going back to original kernel if I do this

```
    unsigned char b = d_input_image[((row * img
    unsigned char g = d_input_image[((row * img
    unsigned char r = d_input_image[((row * img

    d_grayscale_image[row * img_width + col] =
```

Now, it is having a value of 164(**Note:** input bgr values are perfectly fine).

One more thing that I observed, if I remove the round from the kernel, and since image type is float, it is printing the value of 164.5.

Please help me in this, what Should I do in order to have the value as 165

**njuffa** Top Contributor Nov 27 '22

> reddy_v:
>
> Please help me in this, what Should I do in order to have the value as 165

I already addressed that in my previous post. Did you try the little program I posted?

None of the constants 0.299, 0.587. and 0.114 are exactly representable in a binary IEEE-754 floating format, so in conjunction with the rounding that occurs in each floating-point operation and the order of floating-point operations the result can be either 164.5, very slightly smaller than 164.5, or very slightly larger than 164.5. When followed by `round` the result is then going to be either 164 or 165. As my little test program shows (I did not try all possible arrangements of the computation), you *can* re-arrange the computation to match the (somewhat arbitrary) requirement that the result be 165.

> reddy_v:
>
> When I hard code these values in the kernel, output has a value of 165

To my knowledge, there are no guarantees that when a compiler evaluates a compile-time constant floating-point expression that the kind and order of floating-point operations used matches what would happen in a run-time evaluation. Note that I said *a compiler*, so this is not something that applies just to the CUDA compiler.

> reddy_v:
>
> the same way opencv does, and i want these images very accurate

Generally speaking: matches OpenCV != very accurate. You may want to ponder alternative measures of accuracy, e.g. the use of a higher-precision reference. As the famous adage goes: A person with one watch always knows what time it is, a person with two watches can never be sure [Corollary: use a highly accurate reference, such as an atomic clock, to characterize the accuracy of the two watches].

A long time ago I worked on an optimization project with a well-known company in the graphics space. Their reference renderer used double precision throughout, only final results were converted into integer space. So that would be one way of assessing the error in both OpenCV and your code, using some error metric you deem appropriate (e.g. least square error). Caveat: Even in that project issues with (very near) half-way cases cropped up, in that rounding was not always applied consistently to these.

**Robert_Crovella** Moderator Nov 27 '22

> reddy_v:
>
> Please help me in this, what Should I do in order to have the value as 165

I'm pretty much just repeating what njuffa said here already.

If I compile with `--fmad=false` I observe that the calculation changes from 164.499985 to 164.500000 and the rounded result changes from 164 to 165.

As njuffa has already said, this is a possible indication that 164.5 (and therefore 165) *may be a less accurate result* than 164.4999985 and therefore 164.

```
$ cat t1.cu
#include <cstdio>
__global__ void k(unsigned char r, unsigned cha

  float c[3] = {0.299f, 0.587f, 0.114f};
  float p = r*c[0]+g*c[1]+b*c[2];
  printf("float p = %f\n", p);
  unsigned char up = round(p);
  printf("uchar p = %u\n", (unsigned)up);
}

int main(){

  k<<<1,1>>>(172, 160, 168);
  cudaDeviceSynchronize();
}
$ nvcc -o t1 t1.cu
$ ./t1
float p = 164.499985
uchar p = 164
$ nvcc -o t1 t1.cu --fmad=false
$ ./t1
float p = 164.500000
uchar p = 165
$
```

Did you try compiling your code with `--fmad=false` for a test?

## New & Unread Topics

| Topic | Replies | Views | Activity |
|---|---|---|---|
| How to use nvrtc && nvjit? cuda | 3 | 17 | 4h |
| Is there anyway to query the number of remaining operations in a stream? | 3 | 11 | 9h |
| [q] mask in __shfl_sync() cuda kernel | 4 | 15 | 9h |
| Why Does a Variable Become Undefined Within the Same Warp Using Warp Specialization in CUDA? | 2 | 10 | 9h |
| What's the difference between CUDA stack and local memory? | 2 | 17 | 18h |
| Async thrust operation launches appear serially processed in nsight systems cuda performance parallel-computing | 4 | 13 | Aug 30 |
| What is a transaction from HBM to L2? | 2 | 13 | Aug 29 |
| It seems that CUDA kernel are not running parallel | 5 | 14 | Aug 29 |
| GPU1 show C instead of M+C | 1 | 295 | Jan 25 |

Skip to main content

| Topic | Replies | Views | Activity |
|---|---|---|---|
| **Compilation Issues with CUDA 11.5 and GCC 11 on Ubuntu 22.04 - Need help** | 4 | 544 | Apr 9 |

**There are 166 new topics remaining, or browse other topics in**